Object-Oriented and Classical Software Engineering

Eighth Edition, WCB/McGraw-Hill, 2011

Faculty of Natural & Agricultural Sciences
Department of Computer Science and Informatics

UFS•UV

CHAPTER 9

PLANNING AND ESTIMATING

9.1 Planning and the Software Process

- Planning is a continuous process

- The entire project cannot be planned at the start of the process because not enough information is available to plan the entire process

- In some situations, developers are required to produce duration and cost estimates before the specifications are drawn up

Planning and the Software Process (contd)

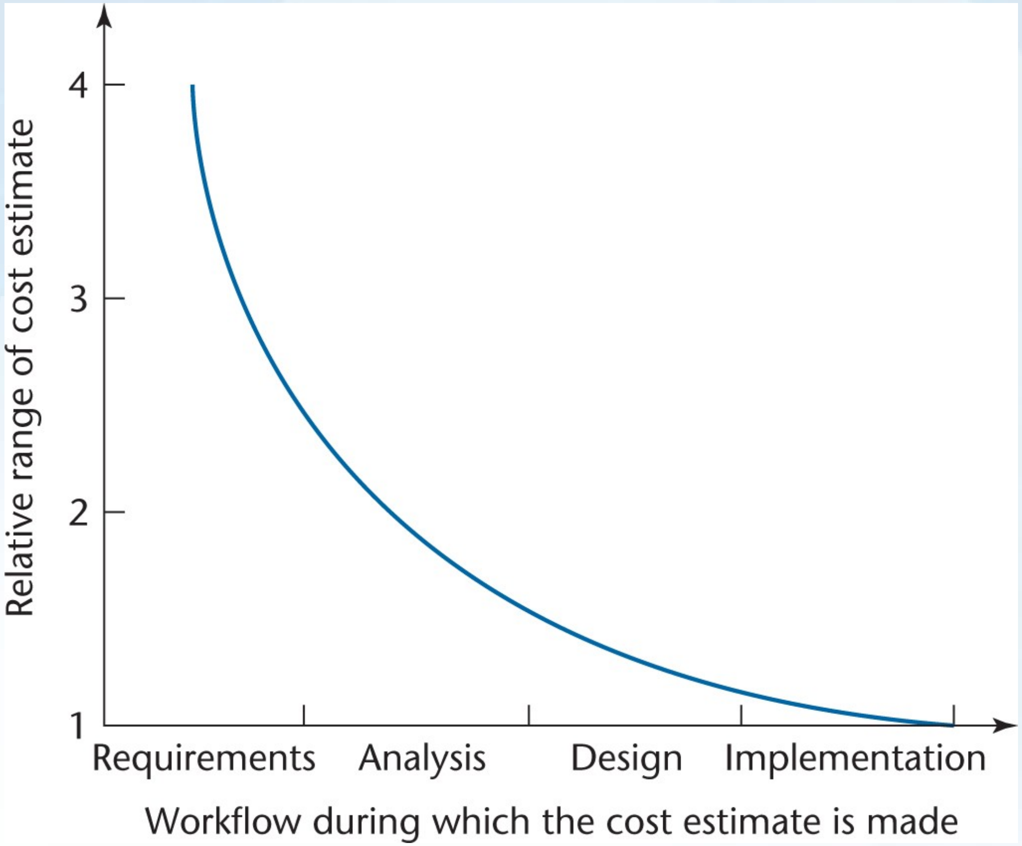

Figure 9.1

- The accuracy of estimation increases as the process proceeds

Planning and the Software Process (contd)

- Example
  - Cost estimate of $1 million during the requirements workflow
    - Likely actual cost is in the range ($0.25M, $4M)
  - Cost estimate of $1 million at the end of the requirements workflow
    - Likely actual cost is in the range ($0.5M, $2M)
  - Cost estimate of $1 million at the end of the analysis workflow (earliest appropriate time)
    - Likely actual cost is in the range ($0.67M, $1.5M)

Planning and the Software Process (contd)

- These four points are shown in the *cone of uncertainty*

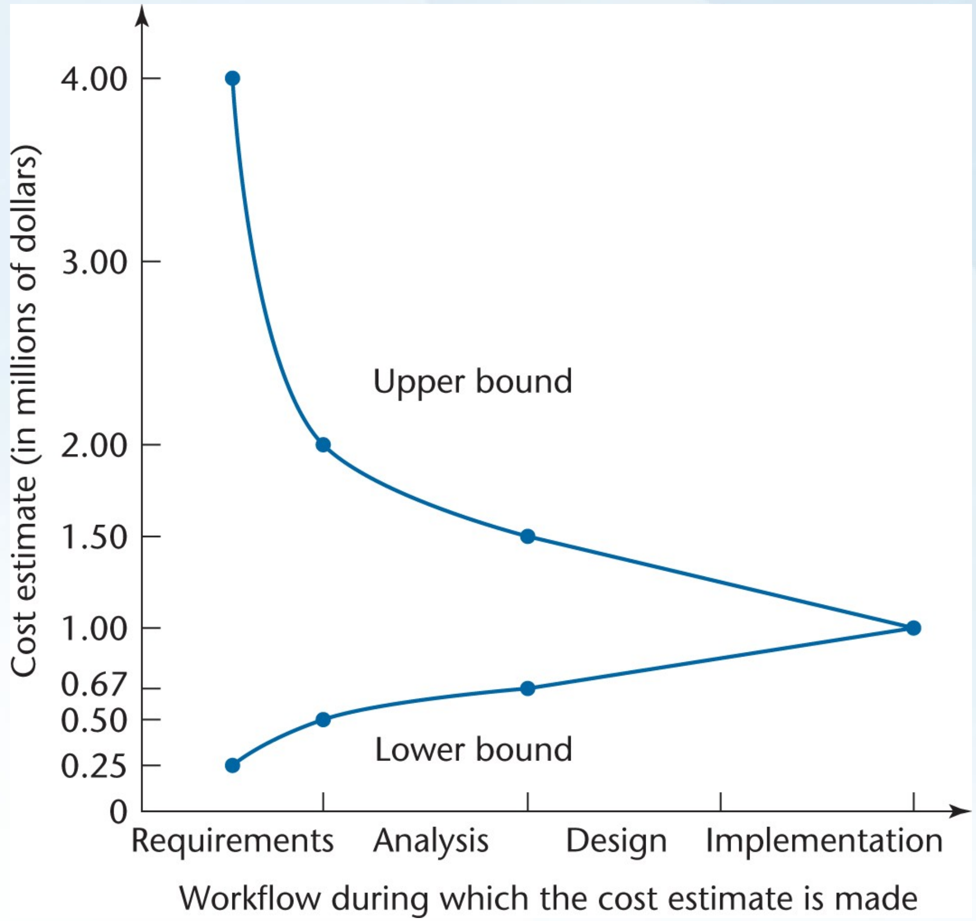

Figure 9.2

9.2 Estimating Duration and Cost

- Accurate cost estimation is a critical part of the software project management plan
  - Internal cost – Cost to the developers
    - Salaries of the development teams, managers, etc.
    - Cost of hardware and software
    - Cost of overhead such as rent, utilities, etc.
  - External cost – Price the client will pay
- Accurate duration estimation is another critical part of any plan
- There are too many variables for accurate estimate of cost or duration, including human factors

9.2.1 Metrics for the Size of a Product

- Lines of code
  - Lines of Code (LOC)
  - Thousand delivered source instructions (KDSI)
- FFP
- Function Points
- COCOMO

Metrics for the Size of a Product - Lines of Code (LOC)

- Alternative metric to LOC is
  - Thousand delivered source instructions (KDSI)
- Problems
  - Creation of source code is only a small part of the total software effort
  - Different languages lead to different numbers of lines of code
  - LOC is not defined for nonprocedural languages (like LISP)

Continues on the next slide

Metrics for the Size of a Product - Lines of Code (LOC)

- Problems (continued)
  - It is not clear how to count lines of code
    - Executable lines of code?
    - Data definitions?
    - Comments?
    - JCL statements (job control language)?
    - Changed/deleted lines?
  - Not everything written is delivered to the client
  - A report, screen, or GUI generator can generate thousands of lines of code in minutes
  - Can only be determined when the product is complete

Metrics for the Size of a Product - Lines of Code (LOC)

- Estimation based on LOC is therefore dangerous
  - To start the estimation process, LOC in the finished product must be estimated
  - The LOC estimate is then used to estimate the cost of the product — an uncertain input to an uncertain cost estimator

9.2.2 Techniques of Cost Estimation

- Expert judgment by similarity (analogy)
- Bottom-up approach
- Algorithmic cost estimation models

Expert Judgment by Analogy

- A number of experts are consulted
- Experts compare the target product to completed products with which they were actively involved

Expert Judgment by Analogy

- The Delphi technique is sometimes needed to achieve consensus
  - Experts work independently
  - Each produce an estimate and rationale for estimate
  - These estimates and rationales are distributed to all experts
  - Experts now produce a second estimate
  - Process continues until a consensus is reached
  - No group meetings take place during the iteration process

Expert Judgment by Analogy

- Problems with Expert Judgment by Analogy
  - Guesses can lead to hopelessly incorrect cost estimates
  - Experts may recollect completed products inaccurately
  - Human experts have biases

- However, the results of estimation by a broad group of experts may be accurate

Bottom-up Approach

- Break the product into smaller components
  - Cost and duration estimates are made for each component and combined for an overall figure
- Advantages
  - Quicker and more accurate
  - Likely to be more detailed
- Weakness
  - A product is more than the sum of its parts
- When using the object-oriented paradigm
  - The independence of the classes helps the bottom-up approach
  - However, the interactions among the classes complicate the estimation process

Algorithmic Cost Estimation Models

- A metric is used as an input to a model for determining product cost
  - An algorithmic model is unbiased, and therefore superior to expert opinion
  - However, estimates are only as good as the underlying assumptions

- COCOMO is a hybrid algorithmic cost estimation model

9.2.3 Intermediate COCOMO

- COnstructive COst MOdel (COCOMO)
- Step 1. Estimate the length of the product in KDSI (Thousand delivered source instructions)
- Step 2. Estimate the product development mode
  - Organic: Small and straightforward
  - Semidetached: Medium sized
  - Embedded: Complex

Intermediate COCOMO

- Step 3. Compute the nominal effort

- Example:
  - Organic product
  - 12,000 delivered source statements (12 KDSI) (estimated)

Nominal effort = $3.2 \times (\text{KDSI})^{1.05}$ person-months
Nominal effort = $3.2 \times (12)^{1.05} = 43$ person-months